4 November – finishing Phase 4

stor01 and GLPI · 15:35 – 18:15 (Germany) · the final file for today, to revise

#	What	Result
1	Step 4 – permanent limits for archive-worker on stor01 (set-property → drop-in files)	✅ **proven after reboot**
2	Incident drill – "deleted the file, but the disk is still full" (lsof +L1)	✅ **500 MB freed**
3	GLPI Incident ticket #4 for the drill	✅ **solved and closed**
4	GLPI L3 Problem behind ticket #3 – archive worker without limits	✅ **solved with Step 4**
5	VMware snapshot 50-processes on all 4 VMs	✅ **Phase 4 complete**

Today in one paragraph: The archive worker on stor01 now has permanent limits – CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20 – written by `systemctl set-property` (without --runtime) as drop-in files on the disk, so they came back after a reboot. In the incident drill a deleted 500 MB file stayed on the disk because a running process still held it open: df showed it, du did not, `lsof +L1` found PID 5468, and stopping it freed the space. Both went into GLPI the ITIL way: Incident ticket #4 for the drill (fix it now) and an L3 Problem for the root cause behind ticket #3 (fix it for good), and a snapshot "50-processes" closes Phase 4.

Inside: **Part 1** Step 4 file (with a refresher of tests 1–3) · **Part 2** incident drill · **Part 3** GLPI ticket, Problem, snapshot.

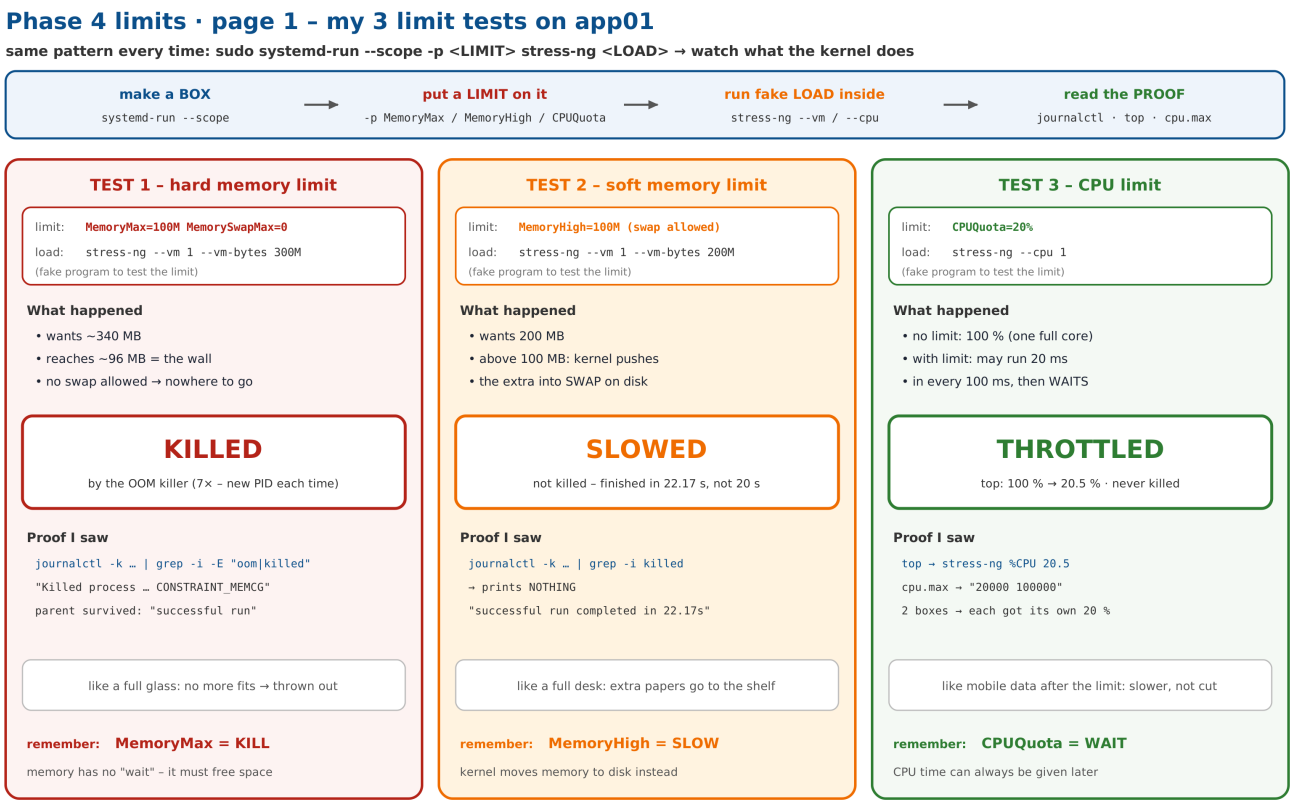
Step 4 on stor01 – permanent limits for archive-worker

Phase 4 · 4 November 2026 · every command with its own screenshot · refresher of tests 1–3 first

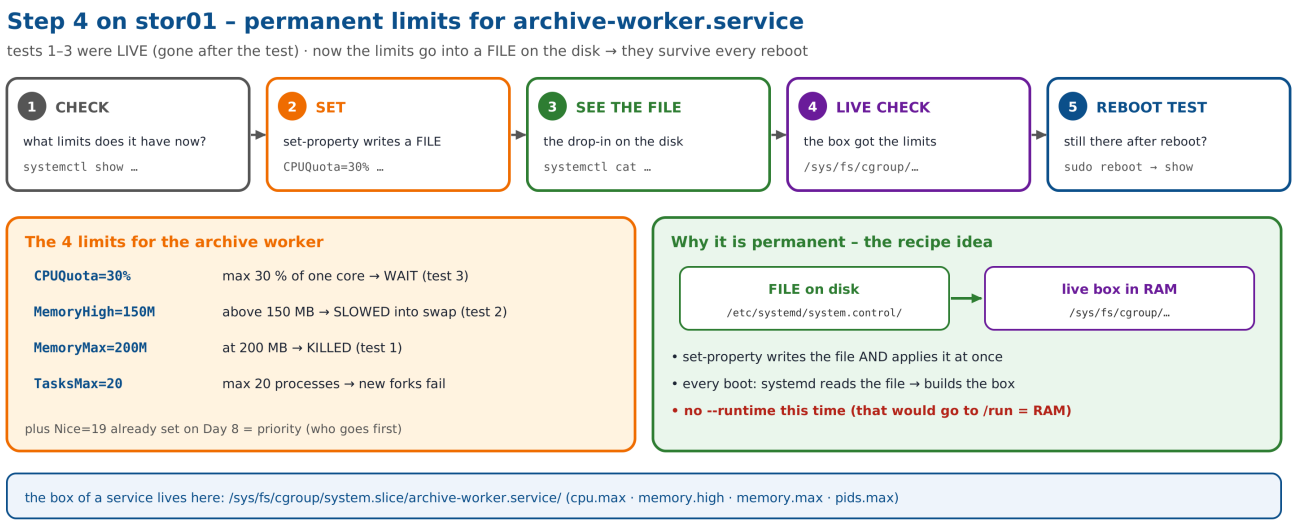
Refresher – what tests 1–3 taught (read this first)

	limit	what happens	remember
Test 1	`MemoryMax=100M`	hard wall – the OOM killer stops the job inside the box	**KILL**
Test 2	`MemoryHigh=100M`	soft limit – extra memory pushed to swap, job slower, not killed	**SLOW**
Test 3	`CPUQuota=20%`	box may run 20 ms of every 100 ms, then waits – throttled	**WAIT**

Tests 1–3 used `systemd-run --scope` = a **temporary** box, a **live change** in RAM – gone when the test ended. Today the limits go on a **real service** and into a **file on the disk** – the "recipe" that systemd reads at every boot. That is what makes them permanent.



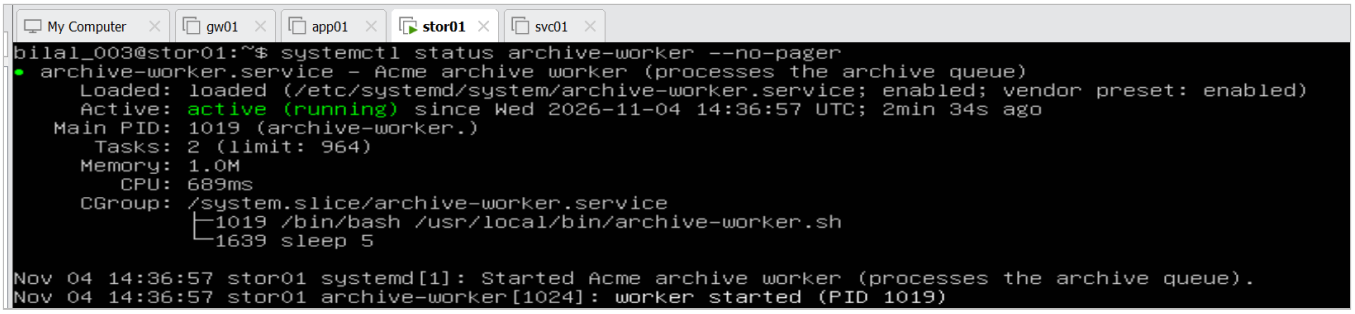
Today's plan – 5 steps



Final paragraph: Tests 1 to 3 were live changes – the temporary boxes and their limits disappeared when each test ended. For a real service the limits must survive a reboot, so they go into a file. sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M MemoryMax=200M TasksMax=20 does two things at once: it applies the limits to the running box and writes them as a drop-in file under /etc/systemd/system.control/ on the disk. At every boot systemd reads that file and builds the box again with the same limits. I check the file with systemctl cat, the live box under /sys/fs/cgroup/system.slice/archive-worker.service/, and prove it is permanent with a reboot. CPUQuota makes the worker wait, MemoryHigh slows it, MemoryMax kills it, TasksMax stops it creating too many processes – and Nice=19 from Day 8 still decides that it goes last.

Step 1 – check what the worker has now

```
systemctl status archive-worker --no-pager
```



What it shows: the worker is **active (running)** since boot (enabled = starts automatically). Its box is `/system.slice/archive-worker.service` with 2 processes inside: the worker script (PID 1019) and the `sleep 5` it runs while waiting for jobs. **Tasks: 2 (limit: 964)** – 964 is the default, not a limit I chose. Memory 1.0 MB – tiny.

```
systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -p
TasksMax -p Nice
```



Before: `infinity` = no CPU or memory limit · TasksMax 964 = the default · `Nice=19` = the low priority I set on Day 8. Priority says *who goes first*; it does not cap anything.

Step 2 – set the 4 limits, permanently

```
sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M
MemoryMax=200M TasksMax=20
```



No output = success. **No `--runtime`** this time – with it the limits would go to `/run` (RAM) and vanish at reboot. Without it, set-property does two things at once: **applies** the limits to the running box and **writes** them to files on the disk.

limit	meaning for the worker
`CPUQuota=30%`	max 30 % of one core – a big archive job cannot take the CPU from NFS or backups (test 3 = wait)
`MemoryHigh=150M`	above 150 MB it is slowed into swap (test 2)
`MemoryMax=200M`	at 200 MB the OOM killer stops it (test 1) – `Restart=on-failure` in its unit file then starts it again
`TasksMax=20`	max 20 processes – a bug that forks endlessly cannot fill the PID pool

Step 3 – see the files on the disk (the recipe)

```
systemctl cat archive-worker
```

```
bilal_003@stor01:~$ systemctl cat archive-worker
# /etc/systemd/system/archive-worker.service
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
```

Part 1 – my original unit file `/etc/systemd/system/archive-worker.service` from Day 8 – unchanged: the script, `Restart=on-failure`, `Nice=19`, `IOSchedulingClass=idle`.

```
# /etc/systemd/system.control/archive-worker.service.d/50-CPUQuota.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
CPUQuota=30%

# /etc/systemd/system.control/archive-worker.service.d/50-MemoryHigh.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
MemoryHigh=157286400

# /etc/systemd/system.control/archive-worker.service.d/50-MemoryMax.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
MemoryMax=209715200

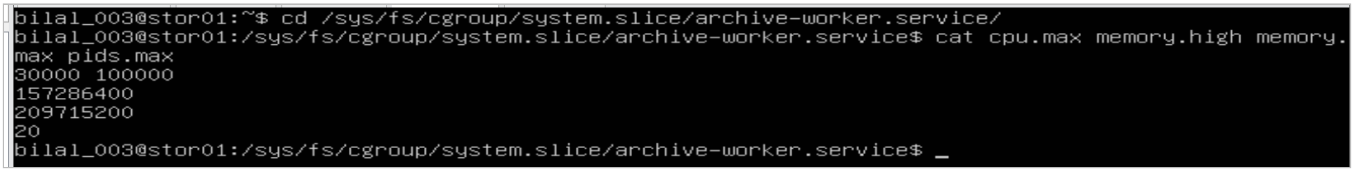
# /etc/systemd/system.control/archive-worker.service.d/50-TasksMax.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
TasksMax=20
bilal_003@stor01:~$ _
```

Part 2 – four new drop-in files in `/etc/systemd/system.control/archive-worker.service.d/`, one per limit (`50-CPUQuota.conf` …). They are added *on top of* the unit file without changing it. "Do not edit" – to change a limit, run set-property again.

Memory is stored in **bytes**: 157286400 = 150 × 1024 × 1024 = 150 MB · 209715200 = 200 MB.

Step 4 – check the live box in the kernel

```
cd /sys/fs/cgroup/system.slice/archive-worker.service/
cat cpu.max memory.high memory.max pids.max
```

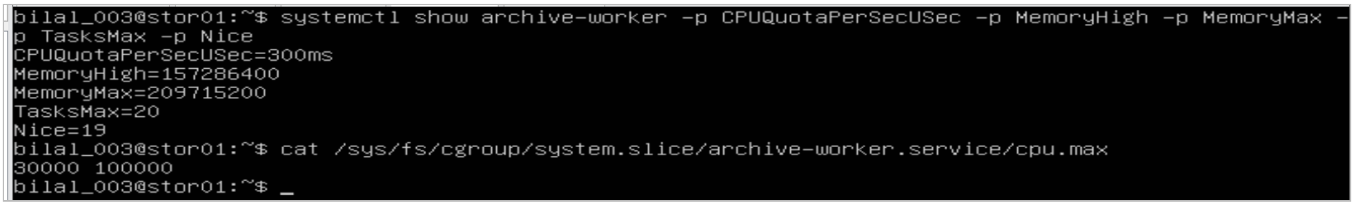


file	value	meaning
`cpu.max`	30000 100000	30 ms of every 100 ms = 30 %
`memory.high`	157286400	150 MB soft limit
`memory.max`	209715200	200 MB hard wall
`pids.max`	20	max 20 processes

The file on the disk (step 3) is the **recipe**; these values in `/sys/fs/cgroup` are the **live box** in RAM that the kernel enforces.

Step 5 – the reboot test (proof that it is permanent)

```
sudo reboot
# after logging in again:
systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -p
TasksMax -p Nice
cat /sys/fs/cgroup/system.slice/archive-worker.service/cpu.max
```



after reboot	meaning
`CPUQuotaPerSecUSec=300ms`	30 % = 300 ms of CPU per second – same limit, new way of writing it
`MemoryHigh=157286400` · `MemoryMax=209715200`	150 MB soft · 200 MB hard – still there
`TasksMax=20` · `Nice=19`	process cap and priority – still there
`cpu.max 30000 100000`	the kernel's live box was rebuilt with the limit

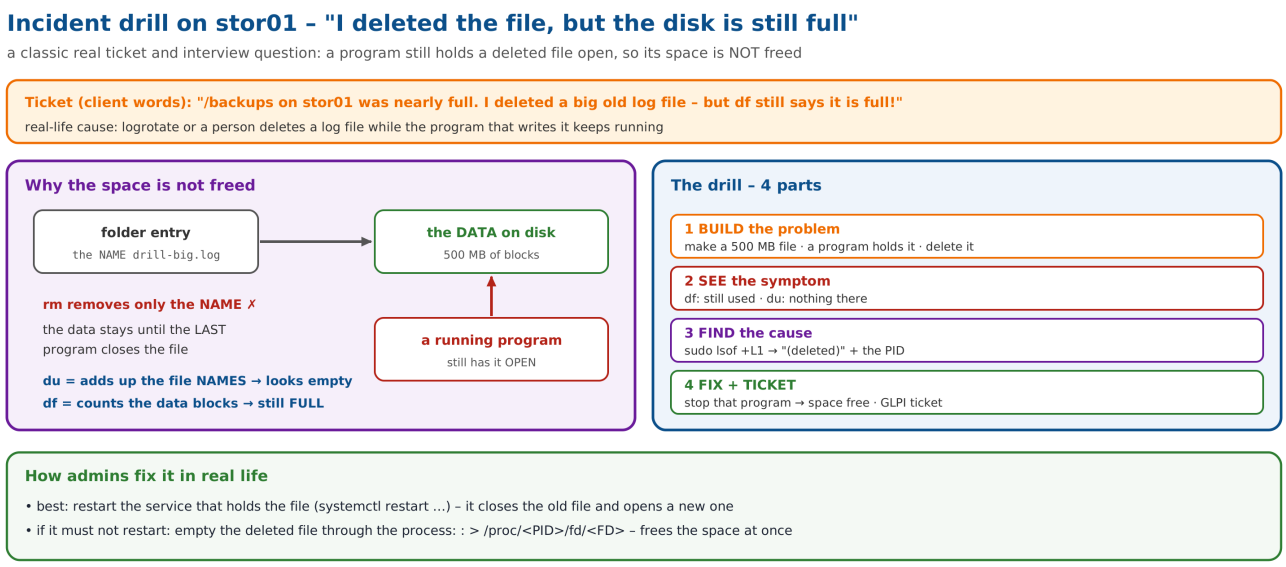
Proof: RAM was emptied by the reboot, yet the box came back with every limit – because systemd read the drop-in files on the disk at boot and built the box again. ✅ Permanent.

Final paragraph: Tests 1 to 3 were live changes – their temporary boxes and limits disappeared when each test ended. For the real archive worker on stor01 the limits must survive a reboot, so they go into files. Before, the worker had no CPU or memory limit

(`infinity`), the default TasksMax 964 and Nice=19 from Day 8. `sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M MemoryMax=200M TasksMax=20` – without `--runtime` – applied the limits at once and wrote four drop-in files under `/etc/systemd/system.control/archive-worker.service.d/`, next to my unchanged unit file; `systemctl cat` shows both. The kernel's live box in `/sys/fs/cgroup/system.slice/archive-worker.service/` confirms `cpu.max 30000 100000`, memory.high 150 MB, memory.max 200 MB and pids.max 20. Now the worker waits above 30 % CPU, is slowed above 150 MB, is killed at 200 MB – and restarted by `Restart=on-failure` – and can never run more than 20 processes, while Nice=19 still makes it go last. At every boot systemd reads the drop-ins and builds the box again: the reboot test proved it: after the restart `systemctl show` still reported 300ms, 150 MB, 200 MB, 20 tasks and Nice=19, and cpu.max was again 30000 100000.

Incident drill on stor01 – "I deleted the file, but the disk is still full"

Phase 4 · 4 November 2026 · a deleted file still held open by a running program · every command with its own marked-up screenshot



Final paragraph: In Linux, deleting a file with rm only removes its name from the folder. The data blocks are freed only when the last program that has the file open closes it. So if a running program still holds a deleted file – typically a log file removed while the service keeps writing – df still counts the space as used, while du, which adds up the file names it can see, shows nothing. The tool to find it is sudo lsof +L1: it lists open files whose name is gone, marked "(deleted)", with the PID and file descriptor of the program holding them. The fix is to restart or stop that program (or empty the file through /proc/<PID>/fd/<FD> if it must keep running), and the space comes back at once. In the drill I build this on stor01's /backups, find it, fix it and record it as a GLPI ticket.

1 – Build the problem

```
df -h /backups
```

```
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G   60K  2.8G   1% /backups
```

① 60K used – empty

① **Before:** /backups (an LVM volume, 2.9 GB) uses only **60K** – it is empty, 1 %.

```
sudo fallocate -l 500M /backups/drill-big.log
```

```
bilal_003@stor01:~$ sudo fallocate -l 500M /backups/drill-big.log
```

No output = success. `fallocate` reserves 500 MB on the disk at once – our big "log file".

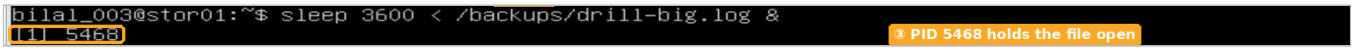
```
df -h /backups
```

```
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  501M  2.3G  18% /backups
```

② +500 MB after fallocate

② **After fallocate: 501M** used, 18 % – the new file takes the 500 MB.

```
sleep 3600 < /backups/drill-big.log &
```



③ **A program holds the file open:** `sleep` uses the file as its input and runs for an hour in the background – job **[1]**, **PID 5468**. It stands in for a real service keeping its log file open.

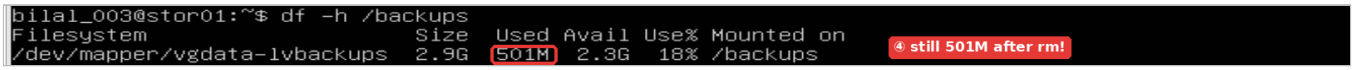
```
sudo rm /backups/drill-big.log
```



No output = the file name is deleted.

2 – See the symptom

```
df -h /backups
```



④ **The problem:** still **501M** used after `rm`! `df` counts the data blocks on the disk – they are still there.

```
sudo du -sh /backups
```



⑤ **The contradiction:** `du` adds up the file names it can find – the name is gone, so it sees only **56K**. When df and du disagree, think "deleted but still open".

3 – Find the cause

```
sudo lsof +L1
```



⑥ **The cause:** `lsof +L1` lists open files that have no name left (fewer than 1 link). Marked: **PID 5468** (sleep) · **524288000** bytes = 500 MB · **(deleted)**. FD `0r` = its input, opened for reading · NLINK `0` = no names left.

4 – Fix it

```
kill 5468
```



Stop the program that holds the file (the PID from lsof).

```
df -h /backups
```

```
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G   60K  2.8G   1% /backups
[1]+  Terminated              sleep 3600 < /backups/drill-big.log
```

⑦ 60K - space free again ✓

⑦ **Fixed:** back to **60K**, 1 % – the kernel freed the 500 MB as soon as the last program closed the file. `[1]+ Terminated` confirms the background job ended.
On a real server I would not kill a service blindly – I would **restart** it (`sudo systemctl restart <service>`) so it closes the old log and opens a new one; if it must keep running, `: > /proc/<PID>/fd/<FD>` empties the deleted file in place.

5 – Record it as a GLPI ticket (text ready to paste)

Title: stor01 – /backups still full after deleting a large log file
Type: Incident · **Category:** Storage · **Urgency:** Medium · **Asset:** stor01
Description: User deleted /backups/drill-big.log (500 MB) but `df -h /backups` still shows 501M used (18 %). `du -sh /backups` shows only 56K.
Diagnosis: `sudo lsof +L1` → process `sleep`, PID 5468, holds /backups/drill-big.log (deleted), 524288000 bytes, NLINK 0. The space is not freed while a process keeps the deleted file open.
Solution: Stopped the holding process (kill 5468). `df -h /backups` → 60K used, 1 %. In production: restart the service that holds the file, or empty it via /proc/<PID>/fd/<FD>.
Prevention: rotate logs with logrotate using `copytruncate` or a service reload, never delete open log files by hand; check `lsof +L1` when df and du disagree.

Final paragraph: On stor01 I created a 500 MB file in /backups, let a running program (sleep, PID 5468) hold it open, and deleted it. `df` still showed 501M used, while `du` found only 56K – because `rm` removes only the file's name; the data blocks stay until the last program that has the file open closes it. `df` counts blocks, `du` counts names, so when they disagree the cause is usually a deleted file that is still open. `sudo lsof +L1` found it: PID 5468, 524288000 bytes, NLINK 0, "(deleted)". Stopping that process freed the space at once (back to 60K). On a real server the right fix is to restart the service holding the file, or empty it via /proc/<PID>/fd/<FD> if it must keep running, and to prevent it with proper log rotation. The incident is recorded in GLPI with symptom, diagnosis, solution and prevention.

Closing Phase 4 – Incident ticket, L3 Problem, snapshot

4 November 2026 · GLPI on svc01 (http://192.168.60.40) · ITIL: Incident = fix it now · Problem = find the root cause, fix it for good

Finishing Phase 4 – incident ticket, L3 Problem, snapshot

ITIL in one line: an INCIDENT = one thing broke, fix it now · a PROBLEM = why does it keep happening? find the root cause, fix it for good

A · Ticket – Incident

today's drill on stor01

"/backups still full after deleting a large log file"

L1/L2 fixes it → status Solved

text ready in the drill file

B · Problem – L3 Engineering

root cause behind ticket #3 (Day 8):

the archive worker had no limits – one big job could starve stor01

solution = Step 4 permanent limits

→ status Solved

C · Snapshot "50-processes"

VMware, on all 4 VMs:

`gw01 · app01 · stor01 · svc01`

a safe point to return to before Phase 5

→ Phase 4 complete

How they link in GLPI

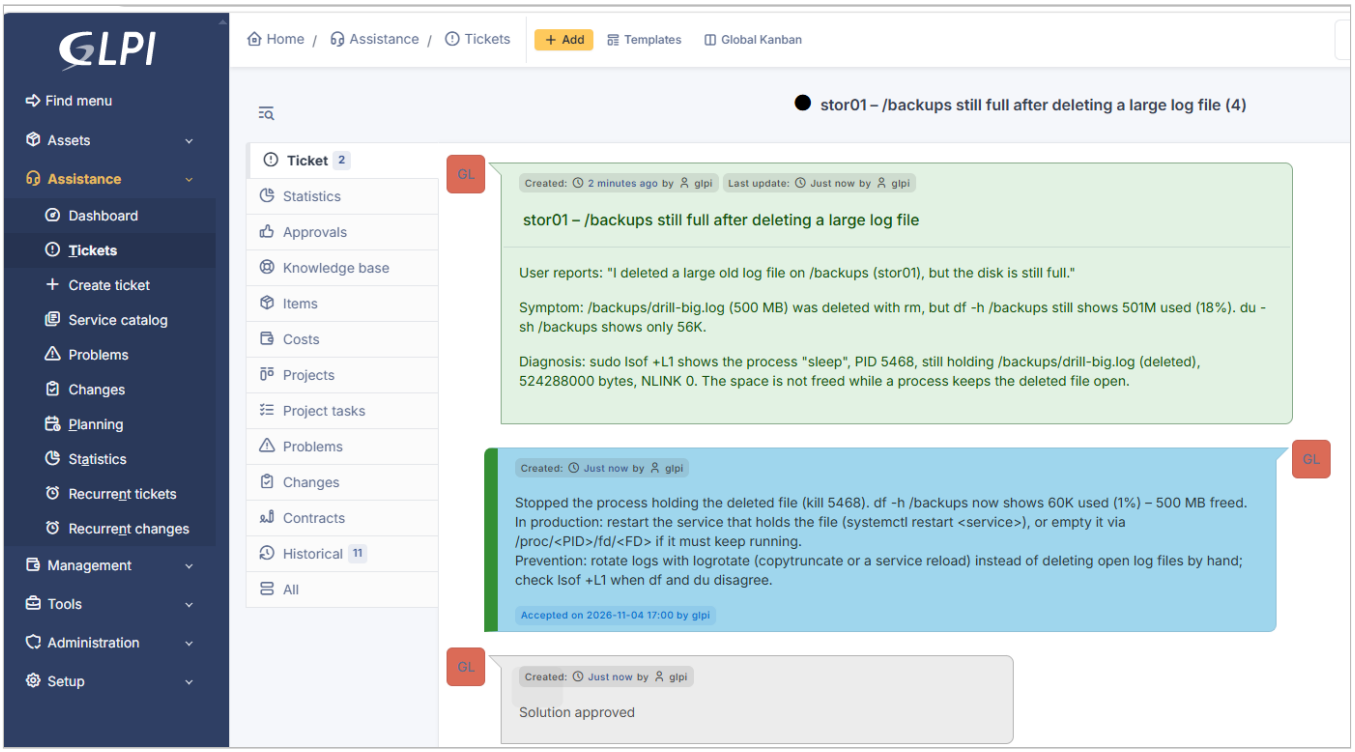
ticket #3 (archive queue, Day 8, L1 → L2) —linked to→ Problem "archive worker without resource limits" (group L3 – Engineering)

the Problem records: symptoms · cause · impact · solution (Step 4) – so the next admin sees WHY and WHAT was changed

Final paragraph: Phase 4 ends with the service-desk side and a safe point. A – today's drill becomes an Incident ticket on stor01: symptom, diagnosis with lsof +L1, solution and prevention, then Solved. B – an Incident is one event; a Problem asks why it keeps happening. The L3 Problem "archive worker without resource limits" is linked to ticket #3 from Day 8, assigned to the group L3 – Engineering, and records the root cause (no limits, so one big archive job could take stor01's CPU and memory) and the permanent solution from Step 4 (CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20, proven after reboot) – then it is Solved. C – a VMware snapshot "50-processes" of all four VMs is a safe point to return to before Phase 5.

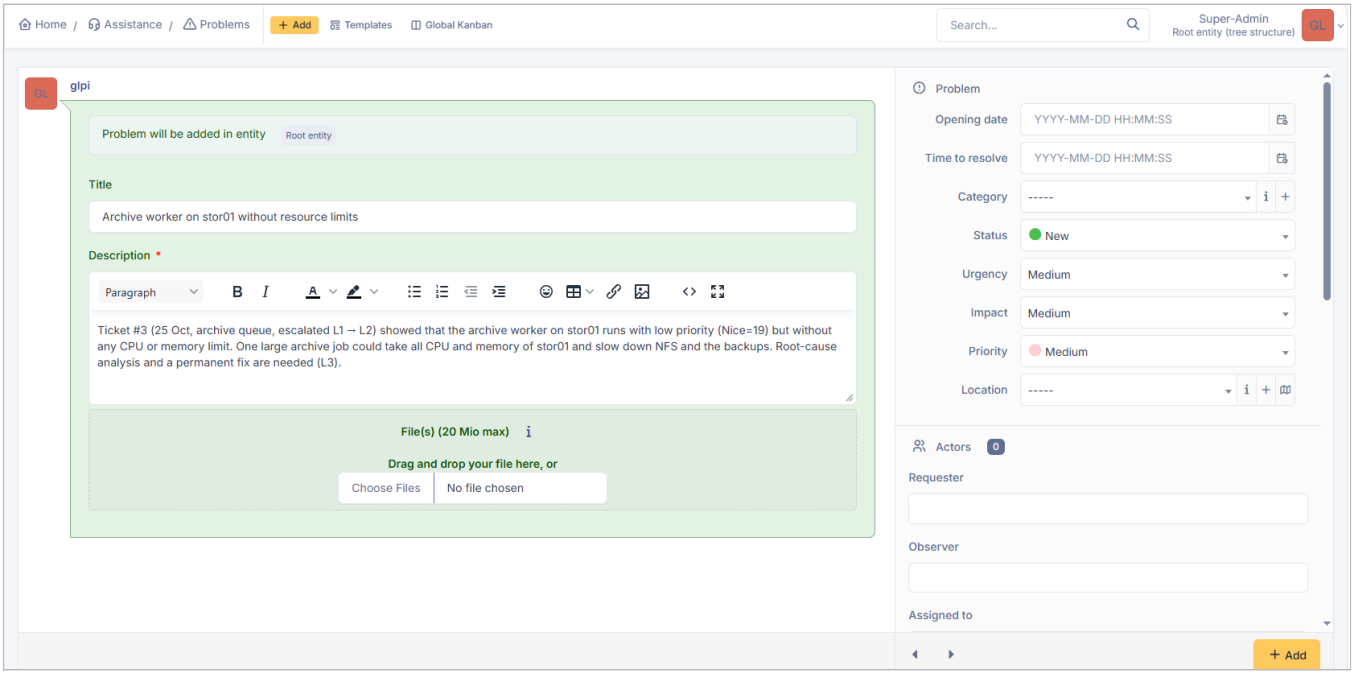
A – Incident ticket #4 for today's drill – Solved and closed

Title *stor01 – /backups still full after deleting a large log file* · Type Incident · Category Storage & Backup · Requester **alice** (Requester = who reported it) · Assigned to group **L2 – Linux Storage** · Item **stor01** (Item = what is broken). The description holds the symptom and the lsof diagnosis; the solution holds the fix and the prevention. The solution was accepted and the ticket closed (black dot). In a company alice would approve it herself.

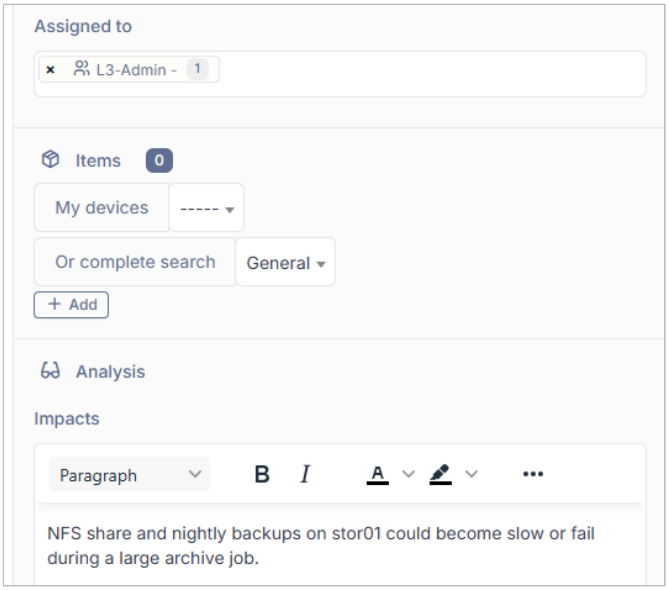


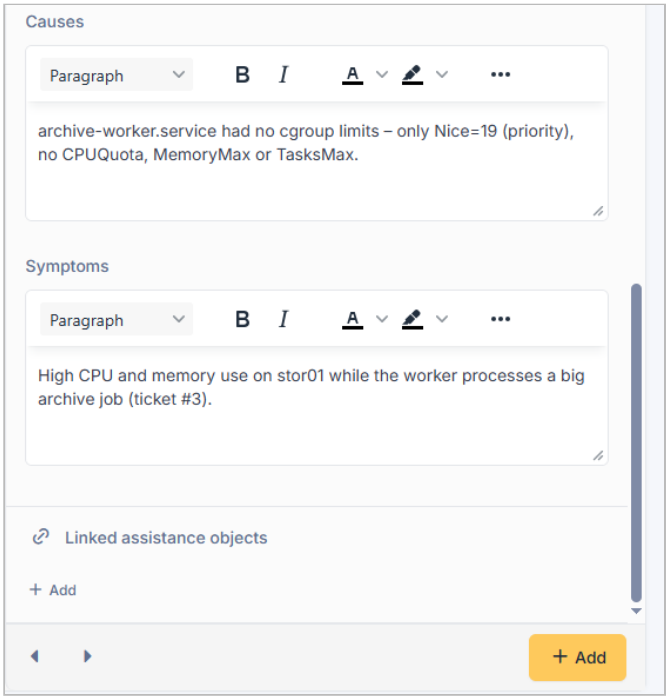
B – L3 Problem: "Archive worker on stor01 without resource limits"

An Incident is one event; a Problem asks why it happens. This Problem is the root cause behind **ticket #3** (25 Oct, archive queue, escalated L1 → L2). Assigned to group **L3-Admin**. Note: it must be created under **Assistance → Problems** – the same text under Tickets would create a ticket.



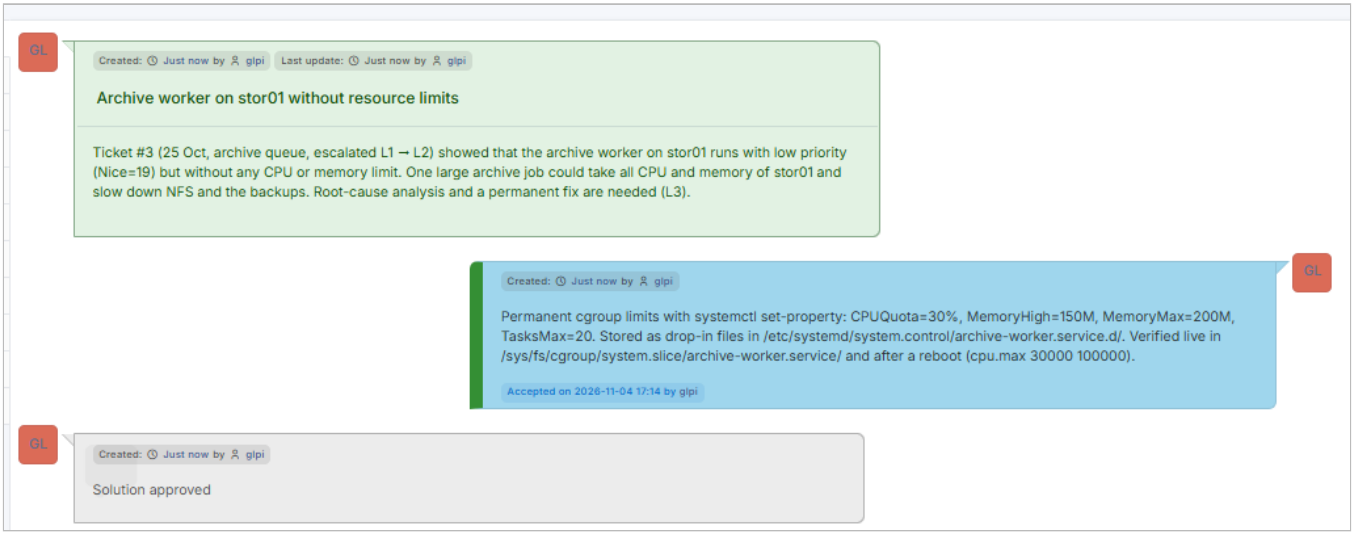
Analysis – the three boxes are part of the Problem form:





Impacts: NFS and backups could become slow · Causes: no cgroup limits, only Nice=19 · Symptoms: high CPU and memory during a big archive job.

B – Solution: the Step 4 limits – approved



The solution records exactly what was changed and how it was verified: `CPUQuota=30%`, `MemoryHigh=150M`, `MemoryMax=200M`, `TasksMax=20`, drop-in files in `/etc/systemd/system.control/archive-worker.service.d/`, checked in `/sys/fs/cgroup` and after a reboot. Ticket #3 and stor01 are linked to the Problem.

C – Snapshot "50-processes"

VMware snapshot **50-processes** on all four VMs (gw01, app01, stor01, svc01) – description *Phase 4 complete: processes, signals, limits, incident drill, L3 Problem*. A safe point to return to before Phase 5.

Final paragraph: Phase 4 ends on the service-desk side. Today's drill became Incident ticket #4: requester alice, group L2 – Linux Storage, item stor01, with the symptom (df 501M, du 56K), the diagnosis (lsof +L1, PID 5468, deleted file still open), the solution and the prevention – solved, approved and closed. Behind ticket #3 from Day 8 I opened an L3 Problem, "Archive worker on stor01 without resource limits", assigned to L3-Admin, with its analysis – impact on NFS and backups, cause no cgroup limits (only Nice=19), symptom high CPU and memory during big jobs – and the permanent solution from Step 4: CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20 as drop-in files, verified after a reboot. An Incident fixes one event; a Problem removes the cause so it does not come back. A snapshot "50-processes" of all four VMs closes Phase 4.